

Digital Logic Assignment Solutions

Assignment 4

December 23, 2025

1 Simplify the following Boolean function and express using NAND/NOR gates

Given: $F(A, B, C, D) = \sum(1, 2, 4, 7, 8, 9, 11) + d(0, 3, 5)$

1.1 a) Simplify the Boolean function

1. Grouping 1-cells (SOP Form)

$AB \backslash CD$	00	01	11	10
00	X	1	X	1
01	1	X	1	0
11	0	0	0	0
10	1	1	1	0

$$F_{SOP} = A'B' + A'C' + B'C' + B'D + A'D$$

2. Grouping 0-cells (POS Form)

$AB \backslash CD$	00	01	11	10
00	X	1	X	1
01	1	X	1	0
11	0	0	0	0
10	1	1	1	0

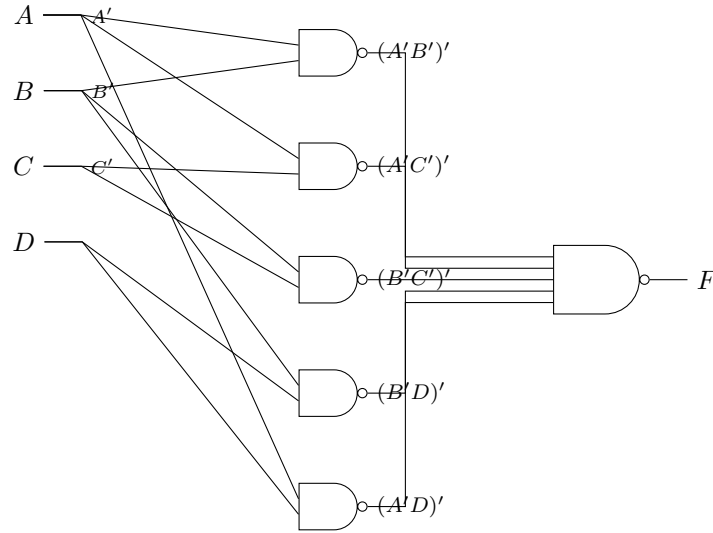
$$\begin{aligned} F' &= AB + BCD' + ACD' \\ F &= (AB)' \cdot (BCD')' \cdot (ACD')' \\ &= (A' + B') \cdot (B' + C' + D) \cdot (A' + C' + D) \end{aligned}$$

1.2 b1) Algebraic method transform (NAND gates only)

$$F = A'B' + A'C' + B'C' + B'D + A'D$$

$$\begin{aligned} F &= ((A'B' + A'C' + B'C' + B'D + A'D)')' \\ &= ((A'B')' \cdot (A'C')' \cdot (B'C')' \cdot (B'D)' \cdot (A'D)')' \end{aligned}$$

1.3 b2) Draw the circuit (NAND gates only)

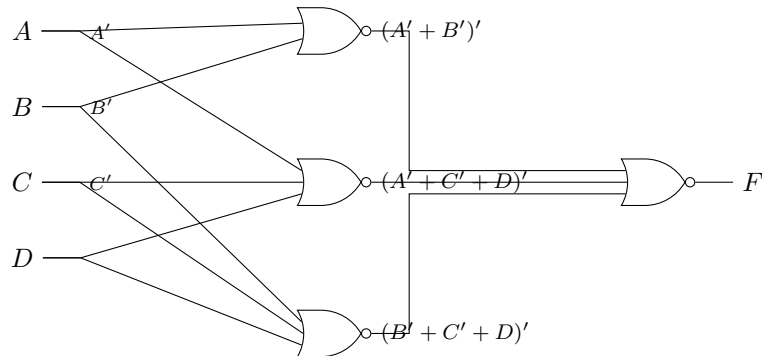


1.4 c1) Algebraic method transform (NOR gates only)

$$F = (A' + B') \cdot (A' + C' + D) \cdot (B' + C' + D)$$

$$\begin{aligned} F &= (((A' + B') \cdot (A' + C' + D) \cdot (B' + C' + D)))' \\ &= ((A' + B')' + (A' + C' + D)' + (B' + C' + D)')' \end{aligned}$$

1.5 c2) Draw the circuit (NOR gates only)

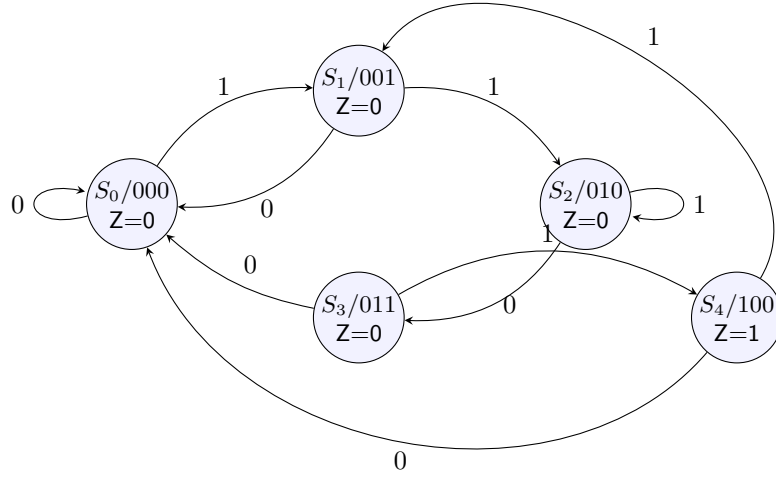


2 Design a combinational circuit with Decoder and NOR gates

2.1 Boolean Function Analysis

Using the principle that $F = \text{NOR}(\text{Maxterms}) = \text{NOR}(\text{Zeros of } F)$:

Func	Expression	SOP (Logic 1)	NOR Inputs (Logic 0)
F_1	$AB + A'B'C'$	$\Sigma m(0, 6, 7)$	D_1, D_2, D_3, D_4, D_5
F_2	$A + B + C'$	$\Sigma m(0, 2, 3, 4, 5, 6, 7)$	D_1 (tied with Logic 0)
F_3	$A'B + AB'$	$\Sigma m(2, 3, 4, 5)$	D_0, D_1, D_6, D_7



3. State Table

Present State		Next State		Output Z
Symbol	$Q_2Q_1Q_0$	$X = 0$	$X = 1$	
S_0	000	000	001	0
S_1	001	000	010	0
S_2	010	011	010	0
S_3	011	000	100	0
S_4	100	000	001	1

3.2 Derivation of Equations (K-Maps)

We derive the JK inputs using the excitation table logic ($0 \rightarrow 0 : 0X, 0 \rightarrow 1 : 1X, 1 \rightarrow 0 : X1, 1 \rightarrow 1 : X0$).

1. Flip-Flop 2 (J_2, K_2)

J_2 Map

$Q_2Q_1 \backslash Q_0$	00	01	11	10
00	0	0	0	0
01	0	0	1	0
11	X	X	X	X
10	X	X	X	X

$$J_2 = Q_1Q_0X$$

K_2 Map

$Q_2Q_1 \backslash Q_0$	00	01	11	10
00	X	X	X	X
01	X	X	X	X
11	X	X	X	X
10	1	1	1	1

$$K_2 = 1$$

2. Flip-Flop 1 (J_1, K_1)

J_1 Map

$Q_2Q_1 \backslash Q_0$	0	1	X	0
0	0	0	1	0
1	X	X	X	X
2	X	X	X	X
3	0	0	X	X

$$J_1 = Q_0X$$

K_1 Map

$Q_2Q_1 \backslash Q_0$	X	X	X	X
0	X	X	1	1
1	X	X	X	X
2	X	X	X	X
3	X	X	X	X

$$K_1 = Q_0$$

3. Flip-Flop 0 (J_0, K_0)

J_0 Map

$Q_2Q_1 \backslash Q_0X$	00	01	11	10
00	0	1	X	X
01	1	0	X	X
11	X	X	X	X
10	0	1	X	X

$$J_0 = Q_1'X + Q_1X' = Q_1 \oplus X$$

K_0 Map

$Q_2Q_1 \backslash Q_0X$	00	01	11	10
00	X	X	1	1
01	X	X	1	1
11	X	X	X	X
10	X	X	X	X

$$K_0 = 1$$

4 Design of Sequence Generator (1011110)

The length of the sequence is $N = 7$. The minimum number of flip-flops must satisfy $N \leq 2^n$.

$$7 \leq 2^3 \implies n \geq 3$$

However, we must check for state conflicts using $n = 3$ (window size 3):

- Sequence: ... **1 1 1** 1 ... (State 111 is followed by 1)
- Sequence: ... **1 1 1** 1 0 ... (State 111 is followed by 0)

Since the state **111** appears twice with different next-bit outputs, $n = 3$ is insufficient. We require minimum $n = 4$ Flip-Flops.

State Transition Table

Time	Target Output Z	Current State				Next State (Shift)
		Q_0	Q_1	Q_2	Q_3	
t_1	1	0	1	1	1	1011
t_2	0	1	0	1	1	0101
t_3	1	0	1	0	1	1010
t_4	1	1	0	1	0	1101
t_5	1	1	1	0	1	1110
t_6	1	1	1	1	0	1111
t_7	0	1	1	1	1	0111

Karnaugh Map Simplification

We map the inputs (Q_0, Q_1, Q_2, Q_3) to the required output Z .

- **Output 0:** States 1011, 1111.
- **Output 1:** States 0111, 0101, 1010, 1101, 1110.
- **Don't Care (X):** All other 9 states (0000, 0001, 0010, 0011, 0100, 0110, 1000, 1001, 1100).

We group the **0s** to solve for Z' .

$Q_0Q_1 \backslash Q_2Q_3$	00	01	11	10
00	X	X	X	X
01	X	1	1	X
11	X	1	0	1
10	X	X	0	1

$Z' = Q_0Q_2Q_3$. Taking the complement for Z :

$$Z = Q_0' + Q_2' + Q_3'$$

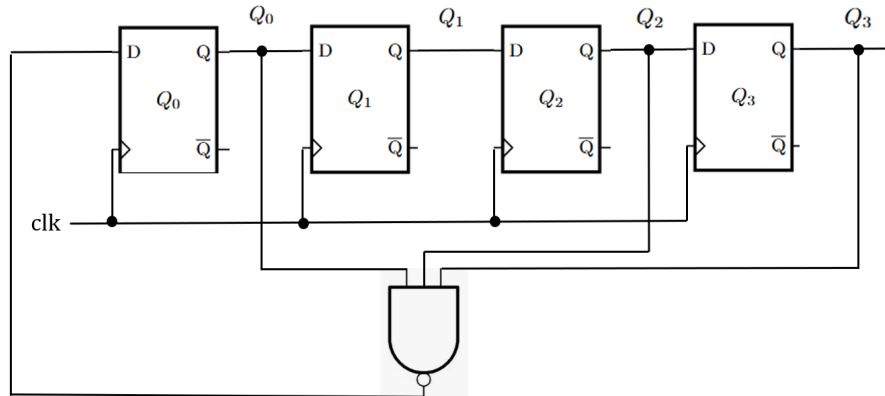


Figure 2: Q4 Logic Diagram

5 Design a Counter with T Flip-Flops (Sequence: 0, 1, 3, 7, 6, 4)

Problem: Design a counter for sequence $0 \rightarrow 1 \rightarrow 3 \rightarrow 7 \rightarrow 6 \rightarrow 4 \rightarrow 0$. Analyze the effect of Don't Care states (2 and 5) and correct the design.

Excitation Table (Sequence Analysis)

Present State ($Q_2Q_1Q_0$)	Next State ($Q_2^+Q_1^+Q_0^+$)	Transition			T Inputs ($Q \oplus Q^+$)		
		Q_2	Q_1	Q_0	T_2	T_1	T_0
0 0 0 (0)	0 0 1 (1)	$0 \rightarrow 0$	$0 \rightarrow 0$	$0 \rightarrow 1$	0	0	1
0 0 1 (1)	0 1 1 (3)	$0 \rightarrow 0$	$0 \rightarrow 1$	$1 \rightarrow 1$	0	1	0
0 1 1 (3)	1 1 1 (7)	$0 \rightarrow 1$	$1 \rightarrow 1$	$1 \rightarrow 1$	1	0	0
1 1 1 (7)	1 1 0 (6)	$1 \rightarrow 1$	$1 \rightarrow 1$	$1 \rightarrow 0$	0	0	1
1 1 0 (6)	1 0 0 (4)	$1 \rightarrow 1$	$1 \rightarrow 0$	$0 \rightarrow 0$	0	1	0
1 0 0 (4)	0 0 0 (0)	$1 \rightarrow 0$	$0 \rightarrow 0$	$0 \rightarrow 0$	1	0	0
0 1 0 (2)	Don't Care				X	X	X
1 0 1 (5)	Don't Care				X	X	X

5.1 a) Input for flip-flop T0 (before correct)

$Q_2 \backslash Q_1Q_0$	00	01	11	10
0	1	0	0	X
1	0	X	1	0

$$T_0 = Q_2'Q_0' + Q_2Q_0 = (Q_2 \oplus Q_0)'$$

5.2 b) Input for flip-flop T1 (before correct)

$Q_2 \backslash Q_1 Q_0$	00	01	11	10
0	0	1	0	X
1	0	X	0	1

$$T_1 = Q_1'Q_0 + Q_1Q_0' = Q_1 \oplus Q_0$$

5.3 c) Input for flip-flop T2 (before correct)

$Q_2 \backslash Q_1 Q_0$	00	01	11	10
0	0	0	1	X
1	1	X	0	0

$$T_2 = Q_2'Q_1 + Q_2Q_1' = Q_2 \oplus Q_1$$

5.4 Explanations and Correction

1. Lock-out Analysis With the equations derived above, let us check the next state for the unused states (010 and 101):

- **State 010** ($Q = 010$): $T_2 = 0 \oplus 1 = 1$; $T_1 = 1 \oplus 0 = 1$; $T_0 = (0 \oplus 0)' = 1$. Next State: $010 \oplus 111 = \mathbf{101}$.
- **State 101** ($Q = 101$): $T_2 = 1 \oplus 0 = 1$; $T_1 = 0 \oplus 1 = 1$; $T_0 = (1 \oplus 1)' = 1$. Next State: $101 \oplus 111 = \mathbf{010}$.

The counter falls into a vicious circle ($010 \leftrightarrow 101$) and never returns to the valid sequence.

2. Correction We force the unused states to reset to **000**. Required transitions: $010 \rightarrow 000$ (Inputs $T = 010$) and $101 \rightarrow 000$ (Inputs $T = 101$). We update the K-maps with specific values instead of Don't Cares:

- T_0 : $m_2(010) = 0, m_5(101) = 1$.
New Eq: $T_0 = Q_2Q_0 + Q_2'Q_1'Q_0'$
- T_1 : $m_2(010) = 1, m_5(101) = 0$.
New Eq: $T_1 = Q_1Q_0' + Q_2'Q_1'Q_0$
- T_2 : $m_2(010) = 0, m_5(101) = 1$.
New Eq: $T_2 = Q_2Q_1' + Q_2'Q_1Q_0$